

Descomposición de Valores Singulares (SVD)

Análisis de reducción de dimensionalidad y agrupación en dataset de estudiantes

Autor: RobertScience Data Consulting
Proyecto: Práctica M29 – Análisis Multivariado
Herramienta: Python / Jupyter Notebook
Entorno: Visual Studio Code

Descripción del proyecto

El objetivo de este análisis es aplicar la técnica de **Descomposición de Valores Singulares (SVD)** a un conjunto de datos de estudiantes, con el fin de reducir la dimensionalidad del dataset y analizar la estructura de agrupación existente entre las observaciones.

A través de este enfoque se busca representar los datos en espacios de dos y tres dimensiones, facilitando la identificación de patrones y similitudes entre los estudiantes en función de su desempeño académico.

Adicionalmente, se evaluará la claridad de las agrupaciones obtenidas y se compararán los resultados con los previamente obtenidos mediante el Análisis de Componentes Principales (PCA).

Importación de librerías

En esta sección se importan las librerías necesarias para la manipulación de datos, visualización y aplicación del algoritmo de descomposición SVD.

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import TruncatedSVD
```

Construcción del conjunto de datos

El dataset utilizado corresponde a un conjunto de estudiantes con calificaciones en distintas materias.

Dado que los datos se proporcionan en formato tabular visual, se procede a reconstruir manualmente el DataFrame para su análisis en Python.

```
In [2]: data = {
    "Estudiante": ["Lucía", "Pedro", "Inés", "Luis", "Andrés", "Ana", "Carlos", "Jo",
    "Matemáticas": [7.0, 7.5, 7.6, 5.0, 6.0, 7.8, 6.3, 7.9, 6.0, 6.8],
    "Ciencias": [6.5, 9.4, 9.2, 6.5, 6.0, 9.6, 6.4, 9.7, 6.0, 7.2],
    "Español": [9.2, 7.3, 8.0, 6.5, 7.8, 7.7, 8.2, 7.5, 6.5, 8.7],
    "Historia": [8.6, 7.0, 8.0, 7.0, 8.9, 8.0, 9.0, 8.0, 5.5, 9.0],
    "EdFisica": [8.0, 7.0, 7.5, 9.0, 7.3, 6.5, 7.2, 6.0, 8.7, 7.0]
}
```

```
df = pd.DataFrame(data)

df
```

Out[2]:

	Estudiante	Matemáticas	Ciencias	Español	Historia	EdFísica
0	Lucía	7.0	6.5	9.2	8.6	8.0
1	Pedro	7.5	9.4	7.3	7.0	7.0
2	Inés	7.6	9.2	8.0	8.0	7.5
3	Luis	5.0	6.5	6.5	7.0	9.0
4	Andrés	6.0	6.0	7.8	8.9	7.3
5	Ana	7.8	9.6	7.7	8.0	6.5
6	Carlos	6.3	6.4	8.2	9.0	7.2
7	José	7.9	9.7	7.5	8.0	6.0
8	Sonia	6.0	6.0	6.5	5.5	8.7
9	María	6.8	7.2	8.7	9.0	7.0

Exploración inicial de los datos

Se realiza una revisión general del dataset para comprender su estructura, tipos de variables y estadísticas descriptivas básicas.

```
In [3]: df.info()

df.describe()

<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Estudiante  10 non-null    str
1   Matemáticas 10 non-null    float64
2   Ciencias    10 non-null    float64
3   Español     10 non-null    float64
4   Historia    10 non-null    float64
5   EdFísica    10 non-null    float64
dtypes: float64(5), str(1)
memory usage: 612.0 bytes
```

Out[3]:

	Matemáticas	Ciencias	Español	Historia	EdFísica
count	10.000000	10.000000	10.000000	10.000000	10.000000
mean	6.790000	7.650000	7.740000	7.900000	7.420000
std	0.951548	1.609865	0.860491	1.121507	0.92832
min	5.000000	6.000000	6.500000	5.500000	6.000000
25%	6.075000	6.425000	7.350000	7.250000	7.000000
50%	6.900000	6.850000	7.750000	8.000000	7.250000
75%	7.575000	9.350000	8.150000	8.825000	7.87500
max	7.900000	9.700000	9.200000	9.000000	9.000000

Selección de variables numéricas

Para aplicar la descomposición SVD se utilizan únicamente las variables numéricas del dataset.

La columna de nombres de estudiantes se conserva únicamente como identificador para la interpretación de resultados.

```
In [4]: X = df.drop(columns=["Estudiante"])

X.head()
```

Out[4]:

	Matemáticas	Ciencias	Español	Historia	EdFísica
0	7.0	6.5	9.2	8.6	8.0
1	7.5	9.4	7.3	7.0	7.0
2	7.6	9.2	8.0	8.0	7.5
3	5.0	6.5	6.5	7.0	9.0
4	6.0	6.0	7.8	8.9	7.3

Estandarización de las variables

Antes de aplicar SVD es necesario estandarizar las variables, ya que el algoritmo es sensible a la escala de los datos.

Se utiliza StandardScaler para asegurar que todas las variables tengan media 0 y desviación estándar 1.

```
In [5]: scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

Aplicación de SVD (reducción a dos dimensiones)

Se aplica la descomposición de valores singulares reduciendo el dataset a dos componentes principales, lo cual permite visualizar la estructura de los datos en un plano bidimensional.

```
In [6]: svd_2 = TruncatedSVD(n_components=2)

X_svd_2 = svd_2.fit_transform(X_scaled)
```

```
In [7]: svd_2.explained_variance_ratio_
```

```
Out[7]: array([0.57864993, 0.32573008])
```

Varianza explicada por los componentes

Se analiza la proporción de varianza explicada por cada uno de los componentes obtenidos mediante SVD.

Esto permite evaluar qué tan representativa es la reducción de dimensionalidad aplicada al dataset original.

```
In [8]: svd_2.explained_variance_ratio_.sum()
```

```
Out[8]: np.float64(0.9043800196790526)
```

Varianza acumulada

Se calcula la varianza acumulada para determinar cuánta información del dataset original se conserva al reducirlo a dos dimensiones.

Visualización de los datos en 2 dimensiones

Se representa gráficamente la proyección de los estudiantes en el espacio reducido a dos dimensiones.

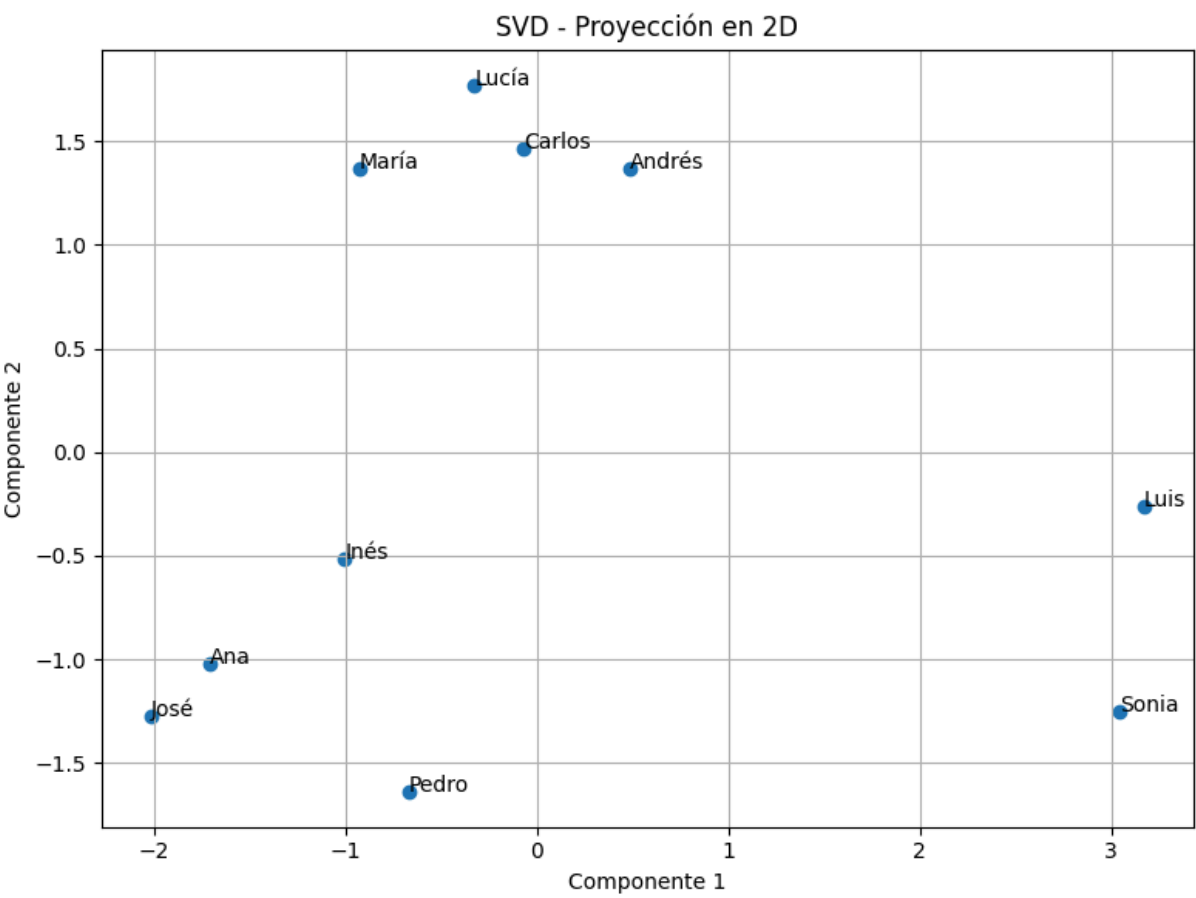
```
In [9]: plt.figure(figsize=(8,6))

plt.scatter(X_svd_2[:,0], X_svd_2[:,1])

for i, name in enumerate(df["Estudiante"]):
    plt.text(X_svd_2[i,0], X_svd_2[i,1], name)

plt.xlabel("Componente 1")
plt.ylabel("Componente 2")
plt.title("SVD - Proyección en 2D")

plt.grid()
plt.tight_layout()
plt.show()
```



Aplicación de SVD (reducción a tres dimensiones)

Se aplica nuevamente la descomposición SVD, esta vez reduciendo el dataset a tres componentes principales. Esto permite analizar la estructura de los datos en un espacio tridimensional y evaluar si la separación entre observaciones mejora respecto a la proyección en 2D.

```
In [10]: svd_3 = TruncatedSVD(n_components=3)

X_svd_3 = svd_3.fit_transform(X_scaled)
```

Visualización de los datos en 3 dimensiones

Se genera una visualización tridimensional para observar la distribución de los estudiantes en el espacio reducido. Esto permite evaluar si existen agrupaciones más claras en comparación con la representación en dos dimensiones.

```
In [11]: from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

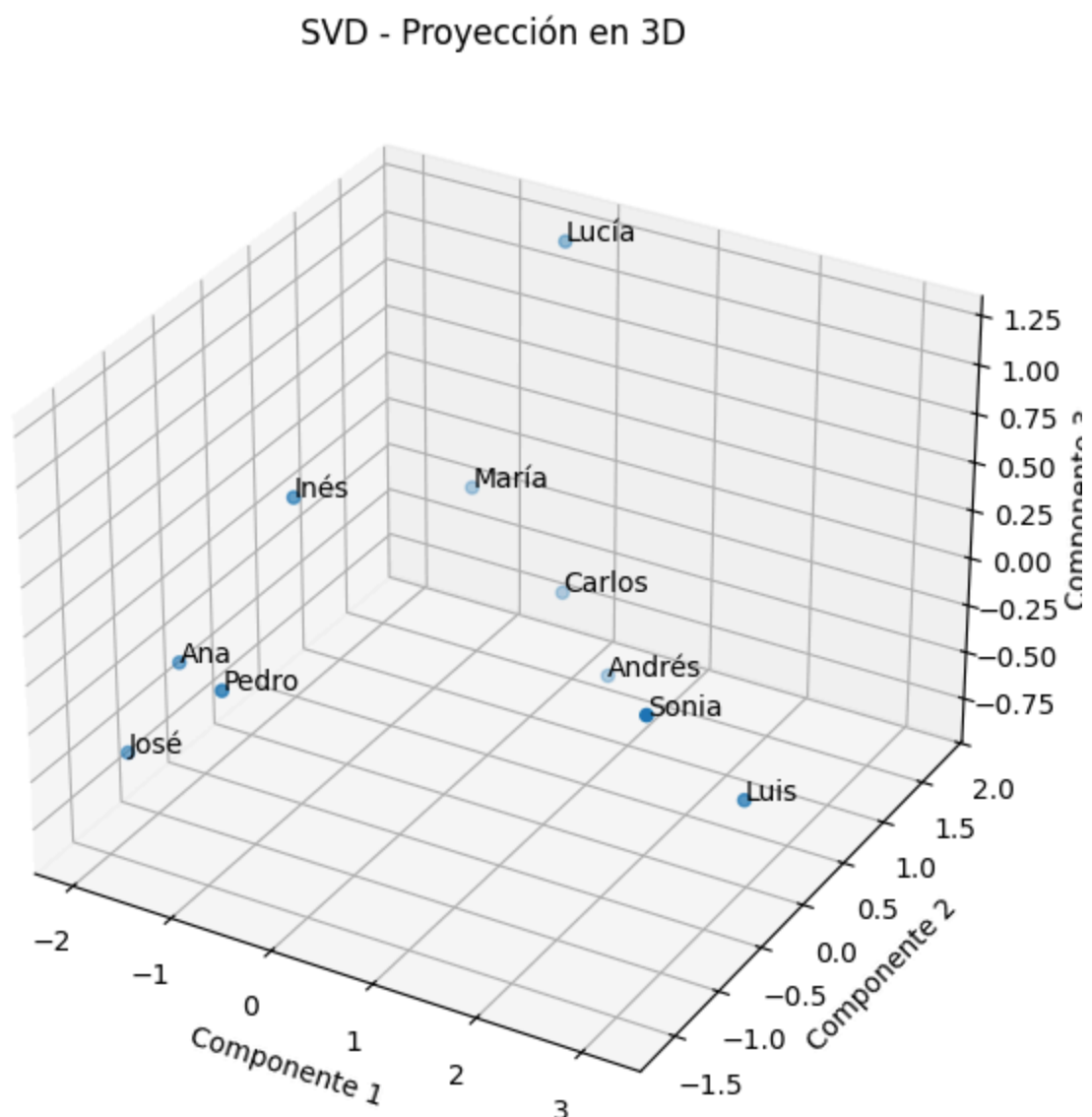
ax.scatter(X_svd_3[:,0], X_svd_3[:,1], X_svd_3[:,2])

for i, name in enumerate(df["Estudiante"]):
    ax.text(X_svd_3[i,0], X_svd_3[i,1], X_svd_3[i,2], name)

ax.set_xlabel("Componente 1")
ax.set_ylabel("Componente 2")
ax.set_zlabel("Componente 3")

ax.set_title("SVD - Proyección en 3D")

plt.tight_layout()
plt.show()
```



Comparación con Análisis de Componentes Principales (PCA)

Para evaluar las diferencias entre SVD y PCA, se aplica PCA al mismo conjunto de datos estandarizado. Esto permite comparar la claridad de agrupación obtenida con ambas técnicas.

```
In [12]: from sklearn.decomposition import PCA

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)
```

Visualización de PCA en 2D

Se representa la proyección de los datos utilizando PCA para comparar visualmente con los resultados obtenidos mediante SVD.

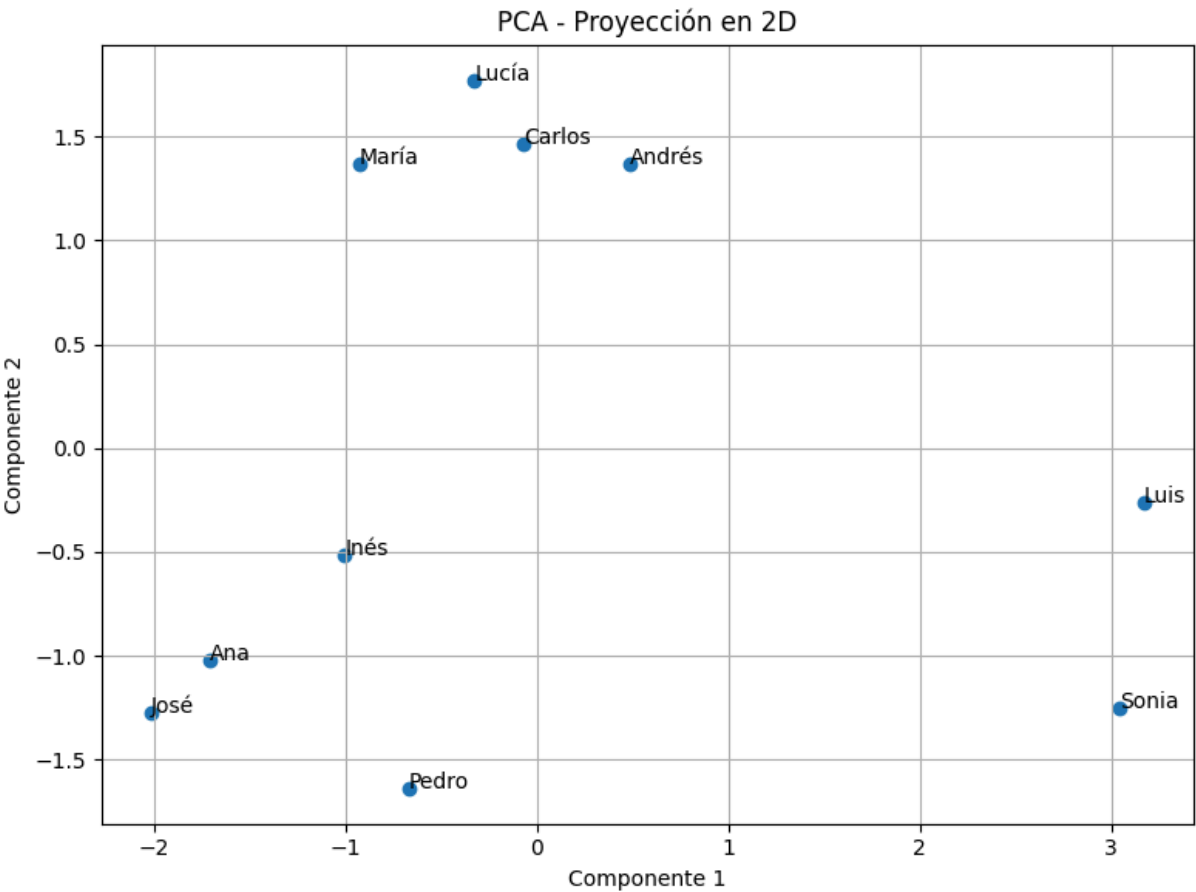
```
In [13]: plt.figure(figsize=(8,6))

plt.scatter(X_pca[:,0], X_pca[:,1])

for i, name in enumerate(df["Estudiante"]):
    plt.text(X_pca[i,0], X_pca[i,1], name)

plt.xlabel("Componente 1")
plt.ylabel("Componente 2")
plt.title("PCA - Proyección en 2D")

plt.grid()
plt.tight_layout()
plt.show()
```



Interpretación de resultados

Al analizar las visualizaciones obtenidas mediante SVD en dos y tres dimensiones, se observa que la representación en tres dimensiones permite una separación ligeramente más clara entre algunos estudiantes, mientras que la proyección en dos dimensiones ya evidencia patrones de agrupación relevantes.

En comparación con PCA, los resultados son consistentes, ya que ambas técnicas buscan representar la mayor variabilidad del dataset en un espacio de menor dimensión. Esto explica por qué las distribuciones observadas presentan estructuras similares.

No obstante, existe una diferencia conceptual importante: SVD realiza una descomposición más general de la matriz de datos, mientras que PCA está orientado específicamente a maximizar la varianza explicada mediante combinaciones lineales ortogonales.

En este análisis, ambas técnicas muestran comportamientos alineados, lo cual indica que la estructura del dataset es estable y puede representarse adecuadamente en espacios reducidos sin pérdida significativa de información.

En conclusión, aunque la proyección en tres dimensiones ofrece una ligera mejora en la separación visual, la representación en dos dimensiones resulta suficiente para identificar patrones generales, y no se observan diferencias significativas entre SVD y PCA debido al tamaño y naturaleza del dataset.

Conclusión

La aplicación de la descomposición de valores singulares permitió reducir la dimensionalidad del dataset de estudiantes, facilitando la visualización de patrones y relaciones entre las observaciones.

La proyección en dos dimensiones resultó suficiente para identificar agrupaciones generales, mientras que la representación en tres dimensiones aportó una ligera mejora en la separación de algunos casos.

Al comparar con PCA, se concluye que ambas técnicas generan resultados consistentes en este tipo de datos, lo que valida su utilidad en el análisis exploratorio.

En conjunto, este ejercicio demuestra cómo las técnicas de reducción de dimensionalidad permiten simplificar la interpretación de datasets multivariados, manteniendo la información más relevante para el análisis.